
```
itinerary =
[0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29,
\30,31,32,33,34,35,36,37,38,39]

np.random.seed(random_seed)
itinerary_ps = itinerary.copy()
for n in range(0,len(itinerary) * 50000):
    itinerary_ps = perturb(cities,itinerary_ps)

print(howfar(genlines(cities,itinerary_ps)))
```

We have just implemented something that might be called a *perturb search* algorithm. It's searching through many thousands of possible itineraries in the hopes of finding a good one, just like a brute force search. However, it's better because while a brute force search would consider every possible itinerary indiscriminately, this is a *guided search* that is considering a set of itineraries that are monotonically decreasing in total traveling distance, so it should arrive at a good solution faster than brute force. We only need to make a few small additions to this perturb search algorithm in order to implement simulated annealing, the capstone algorithm of this chapter.

Before we jump into the code for simulated annealing, we'll go over what kind of improvement it offers over the algorithms we've discussed so far. We also want to introduce a temperature function that allows us to implement the features of simulated annealing in Python.

Algorithms for the Avaricious

The nearest neighbor and perturb search algorithms that we've considered so far belong to a class of algorithms called *greedy* algorithms. Greedy algorithms proceed in steps, and they make choices that are locally optimal at each step but may not be globally optimal once all the steps are considered. In the case of our nearest neighbor algorithm, at each step, we look for the closest city to where we are at that step, without any regard to the rest of the cities. Visiting the closest city is locally optimal because it minimizes the distance we travel at the step we're on. However, since it doesn't take into account all cities at once, it may not be globally optimal—it may lead us to take strange paths around the map that eventually make